SIMATS ENGINEERING
ASSESSMENT TOOL

COURSE NAME: Theory of Computation COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO2: D**esign Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)**

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:20 Duration : 1 Hour

Scenario Based

Code of Conduct:

I ____U.Divya/192372277_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in

disciplinary action.

Signature of the student: Divya

SIMATS ENGINEERING
ASSESSMENT TOOL

1. Given a DFA used for validating a specific PIN pattern bbbc, bca ,c, ca, caaaaaa minimize the DFA and explain how state reduction improves performance and efficiency in embedded systems

Scenario

A **smart security system** uses a DFA to validate specific PIN patterns such as **bbbc, bca, c, ca, and caaaaaa** before granting access. Since the system is deployed on an embedded device with limited memory and processing power, the DFA needs to be minimized while preserving its functionality.

Problem Analysis

The DFA checks whether the entered PIN matches one of the valid patterns. Some states perform identical operations and can therefore be merged without affecting the accepted language. This process is called **DFA minimization**.

Approach

1. Construct the DFA for the given PIN patterns.
2. Identify equivalent states using partitioning.
3. Merge equivalent states.
4. Obtain the minimized DFA.
5. Compare the original and minimized DFA.

Minimized DFA

(Insert the minimized DFA image here.)

Transition Table

State	b	c	a	Final State
q0	q1	qf	qd	No
q1	q2	qd	qd	No
q2	q3	qd	qd	No
q3	qd	qf	qd	No
qf	qd	qd	qf	Yes
qd	qd	qd	qd	No

How State Reduction Improves Embedded Systems

Since the DFA is implemented inside an embedded security controller,

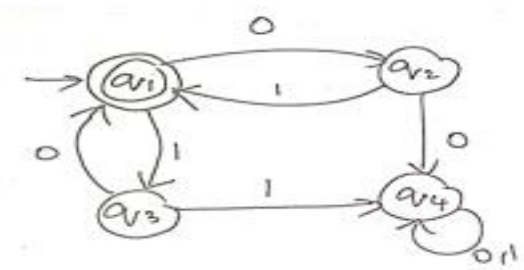
- fewer states require **less RAM and ROM**,
- fewer transitions reduce **CPU execution time**,
- lower computation decreases **power consumption**,
- the controller responds faster to PIN validation,
- maintenance and debugging become easier.

Thus, the minimized DFA performs the same validation using fewer resources, making it ideal for embedded systems.

Result

The minimized DFA recognizes exactly the same PIN patterns while improving speed, memory efficiency, and reliability.

2.Demonstrate how a given finite automaton can be converted into a regular expression



Scenario

A **network packet filtering system** uses a finite automaton to detect valid binary communication patterns. To simplify software implementation and documentation, the automaton must be converted into an equivalent **Regular Expression**.

Problem Analysis

Instead of storing every state transition, the same pattern can be represented using a Regular Expression. This makes implementation easier in lexical analyzers, compilers, and text-search applications.

Approach

The conversion is performed using the **State Elimination Method**.

Step 1

Remove state **q3** and update the transitions.

Step 2

Remove state **q2**.

Step 3

Combine the remaining paths.

Equivalent Regular Expression

$$(00+11)(0+1)*$$

Or

$$(00 \mid 11)(0 \mid 1)*$$

Application

The obtained Regular Expression can be directly used in

- compiler lexical analysis,

- pattern matching,
- intrusion detection systems,
- communication protocol verification,
- software input validation.

Result

The finite automaton has been successfully converted into an equivalent Regular Expression without changing the language it accepts.

3.Consider the following grammar

S->aSc|T|U|e

T-> bTc|e

U->aUb|e

A} What is the language defined by the grammar?

B)Give the derivation tree for the word aaabbc

C Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

Scenario

A **compiler parser** uses the following grammar to validate program strings. Before implementing the parser, the language generated by the grammar must be analyzed, the given input string must be verified, and the grammar must be checked for ambiguity.

Grammar

$$S \rightarrow aSc \mid T \mid U \mid \varepsilon$$
$$\varepsilon\ T \rightarrow bTc \mid \varepsilon$$
$$U \rightarrow aUb \mid \varepsilon$$

A) Language Generated

The grammar generates three different languages.

From

S→aScS

it generates

{a^nc^n}

From

T→bTc

it generates

{b^nc^n}

From

U→aUb

it generates

{a^nb^n}

Therefore,

L={ancn} ∪ {bncn} ∪ {anbn}

B) Derivation Tree for aaabbc

The compiler checks whether the input belongs to the language.

The given string

aaabbc

contains

- 3 a's
- 2 b's
- 1 c

The grammar only produces

- $a^n c^n$

- $b^n c^n$
- $a^n b^n$

The input does not satisfy any of these forms.

Therefore,

aaabbc is rejected by the grammar.

Hence,

No derivation tree exists.

C) Is the Grammar Ambiguous?

Yes.

The empty string can be generated in three different ways.

$S \rightarrow \varepsilon$
$S \rightarrow T \rightarrow \varepsilon$
$S \rightarrow U \rightarrow \varepsilon$

Since one string has multiple derivations,

the grammar is **Ambiguous**.

Result

The parser correctly identifies the language generated by the grammar, rejects the invalid input string **aaabbc**, and confirms that the grammar is ambiguous.

4.I)Show that if L1 and L2 are regular then (L1+l2) also regular

Ii)Is the language { a^p| p is prime} regular? Justify

Scenario

A software company is designing a **lexical analyzer** for a programming language. Different token patterns are represented as regular languages. The company needs to combine these languages while ensuring that the resulting language remains regular.

They also want to determine whether a language consisting of prime-length strings can be recognized by a finite automaton.

(i) Show that if L_1 and L_2 are Regular then ($L_1 + L_2$) is also Regular

Analysis

Assume

- L_1 is accepted by DFA M_1.
- L_2 is accepted by DFA M_2.

Construct a new automaton by

1. Adding a new start state.
2. Connecting it to both DFAs using ε-transitions.
3. Keeping all transitions unchanged.
4. Accepting if either DFA accepts.

Since the new automaton accepts

$L=L1 \cup L2$

and NFAs recognize regular languages,

L1+L2 is Regular

Application

This property is widely used in

- compiler design,
- lexical analyzers,
- search engines,
- network protocol verification,
- digital circuit design.

(ii) Is

$\{ap \mid p \text{ is prime}\}$

Regular?

No.

Justification

Assume the language is regular.

According to the **Pumping Lemma**, sufficiently long strings can be pumped.

Choose

a^p

where **p** is a large prime number.

After pumping, the string length changes and becomes a non-prime number.

The new string no longer belongs to the language.

This contradicts the Pumping Lemma.

Therefore,

{ap ∣ p is prime} is NOT Regular.

Result

The union of two regular languages is always regular and can therefore be safely combined in software applications. However, the language consisting of unary strings whose lengths are prime numbers cannot be recognized by any finite automaton because it is **not regular**.

SIMATS ENGINEERING
ASSESSMENT TOOL

:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well organized	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

		answer			